

Student Portal — Complete Build Guide

From the student-portal-pipeline.zip to a defended, deployed project

Akwenwi Kenzel Che Niba | ICT University, Yaoundé

Start: 16 September 2026 | Delivery: 1 October 2026

How to use this guide. Each part below is a service or phase from the project plan, in build order. Every part tells you exactly which files to create or edit inside the `student-portal-pipeline` folder, the commands to run, and what result confirms you're done before moving on. Follow it in order — later services depend on earlier ones being deployed and reachable.

Contents

Part	Covers	Target dates
0	Unblock the pipeline (Docker socket fix)	16 Sep
1	Auth Service	16–18 Sep
2	Student Service + Dashboard/BFF	19–21 Sep
3	Transcript + Past Papers + Notification	22–25 Sep
4	Full-system load test at 1,000 users	26–28 Sep
5	Documentation, polish, rehearsal	29–30 Sep
6	Final checks and defence day	1 Oct

Part 0 — Unblock the Pipeline

Goal: prove a request can travel gateway → container → back, before anything else is built.

Step 1 — Open the project

In File Explorer, open the `student-portal-pipeline` folder (the one containing `docker-compose.yml`), click the address bar, type `powershell`, press Enter.

Step 2 — Run the socket diagnostic

```
.\diagnose-socket.ps1
```

Read the output under **"3. THE KEY TEST"**. JSON back means the socket already works — skip to Step 4. Nothing, or an error, means continue to Step 3.

Step 3 — Fix Docker socket access (if Test 3 failed)

1. Docker Desktop → **Settings** → **Advanced** → enable **"Allow the default Docker socket to be used."**
2. **Settings** → **General** → confirm **"Use the WSL 2 based engine"** is checked.
3. Click **Apply & Restart**.
4. Right-click the Docker whale icon in the Windows system tray → **Quit Docker Desktop** (closing the window is not enough).
5. Reopen Docker Desktop from the Start menu, wait until the tray icon stops animating.
6. Re-run `.\diagnose-socket.ps1` — Test 3 should now return JSON.

Step 4 — Bring the pipeline up

```
docker compose down  
docker compose up -d --build
```

Step 5 — Confirm it works

```
curl http://localhost/health
```

Expected: `{"status":"ok","service":"health-service",...}`. Then open `http://localhost:8080/dashboard/` in a browser, click **HTTP Routers**, and confirm a router named **health** is listed.

Exit criterion: curl returns JSON and the health router appears. Do not start Part 1 until both are true.

Part 1 — Auth Service (16–18 September)

Everything else depends on this service, so it is built and load-tested alone before anything else touches it. This part also establishes the template you'll repeat for every later service — read it closely even if you skim later parts.

1.1 — Create the service folder

Inside `services/`, create a new folder `auth-service/` with the same three files as `health-service/`:

File	Purpose
<code>package.json</code>	Dependencies — add <code>bcrypt/argon2</code> , <code>jsonwebtoken</code> , <code>pg</code> , <code>ioredis</code> alongside <code>express</code>
<code>Dockerfile</code>	Copy <code>health-service</code> 's <code>Dockerfile</code> verbatim — same Node base image, same structure
<code>index.js</code>	The actual Auth logic — endpoints below

1.2 — Design the schema

The auth schema and an `auth_svc` role already exist from `init-db/01-schemas.sql`. Inside that schema, create at minimum:

```
users
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  institution_id UUID NOT NULL DEFAULT '00000000-0000-0000-0000-000000000001',
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  role TEXT NOT NULL CHECK (role IN ('student', 'lecturer', 'admin')),
  created_at TIMESTAMPTZ DEFAULT now()

refresh_tokens
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  token_hash TEXT NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL,
  revoked BOOLEAN DEFAULT false
```

Every table carries `institution_id` from day one, per the SaaS note in the original plan — it costs nothing now and avoids a painful migration later.

1.3 — Build the endpoints

Endpoint	What it does
POST <code>/auth/register</code>	Create a user, hash the password (<code>bcrypt</code> or <code>argon2</code> — never plaintext)
POST <code>/auth/login</code>	Verify credentials, issue a JWT access token + refresh token
POST <code>/auth/refresh</code>	Exchange a valid refresh token for a new access token
POST <code>/auth/logout</code>	Revoke the refresh token
POST <code>/auth/password-reset/request</code>	Issue a reset token (log it to console for now — real email comes via Notification Service in Part 3)
POST <code>/auth/password-reset/confirm</code>	Consume the token, set a new password

1.4 — Connect through PgBouncer, not Postgres directly

This is the single most important line in this whole service. In your database connection string, point at PgBouncer's port, never Postgres's:

```
DATABASE_URL=postgres://auth_svc:auth_dev_pw@pgbouncer:6432/student_portal
```

Why this matters: PgBouncer pools connections so Postgres doesn't choke when 1,000 students try to log in at once. Connecting straight to postgres:5432 instead is the single most common reason this kind of system crashes under load. Get this right now — it's much harder to catch during Part 4's load test.

1.5 — Store sessions in Redis

Cache active refresh tokens or session state in Redis (redis:6379 on the same network) rather than hitting Postgres on every authenticated request.

1.6 — Wire it into the pipeline

Add this block to docker-compose.yml, under services:, copying the shape of health-service:

```
auth-service:
  build: ./services/auth-service
  container_name: portal-auth
  restart: unless-stopped
  environment:
    DATABASE_URL: postgres://auth_svc:auth_dev_pw@pgbouncer:6432/student_portal
    REDIS_URL: redis://redis:6379
    JWT_SECRET: ${JWT_SECRET:-dev_secret_change_me}
  networks:
    - portal-net
  labels:
    - "traefik.enable=true"
    - "traefik.http.routers.auth.rule=PathPrefix(`/auth`)"
    - "traefik.http.routers.auth.entrypoints=web"
    - "traefik.http.services.auth.loadbalancer.server.port=3000"
  depends_on:
    - gateway
    - pgbouncer
    - redis
```

1.7 — Add it to CI

In .github/workflows/ci.yml, add auth-service to the matrix.service list alongside health-service.

1.8 — Document and test

1. Write Swagger/OpenAPI comments for each endpoint now, while the logic is fresh — retrofitting docs later always gets skipped under deadline pressure.
2. Rebuild and start: `docker compose up -d --build`
3. Manually test through the gateway: `curl -X POST http://localhost/auth/register -H "Content-Type: application/json" -d "{...}"`
4. Write a first k6 script targeting `http://localhost/auth/login`.

5. Run it at 100, then 500 virtual users. Record response times and error rate — this is your Phase 1 baseline, referenced again in Part 4.

Exit criterion: a user can register and log in through the gateway, and you have a saved k6 report for Auth alone at 100 and 500 VUs.

Part 2 — Student Service + Dashboard/BFF (19–21 September)

2.1 — Student Service

Create `services/student-service/`, same three-file shape as Auth. Uses the `student` schema and `student_svc` role already in `init-db/01-schemas.sql`, connected through PgBouncer exactly as Auth was.

Table	Holds
profiles	Name, contact info, institution_id, enrolment status
enrolments	Programme, department, year of study, status

Endpoints: `GET /student/profile/:id`, `PUT /student/profile/:id`, `GET /student/enrolment/:id`.
Traefik label rule: `PathPrefix(`/student`)`, same pattern as 1.6.

2.2 — Dashboard/BFF Service

Create `services/dashboard-service/`. Unlike the others, it has **no database schema of its own** — it calls Auth and Student internally and composes one response.

```
GET /dashboard/home
→ calls http://auth-service:3000/auth/verify (validate the token)
→ calls http://student-service:3000/student/profile/:id
→ merges both into one JSON payload for the frontend
```

Route: `PathPrefix(`/dashboard`)`. Cache the composed payload in Redis with a short TTL (a few seconds) — this endpoint gets hit on every page load, so it's the highest-value place to cache in the whole system.

2.3 — Stand up Prometheus + Grafana now

Add to `docker-compose.yml`:

```
prometheus:
  image: prom/prometheus:latest
  container_name: portal-prometheus
  volumes:
    - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml:ro
  ports:
    - "9090:9090"
  networks:
    - portal-net

grafana:
  image: grafana/grafana:latest
  container_name: portal-grafana
  ports:
    - "3001:3000"
  networks:
    - portal-net
  depends_on:
    - prometheus
```

Create `monitoring/prometheus.yml` with a scrape target for each service's `/metrics` endpoint (add basic Express metrics middleware to each service as you build it, e.g. `prom-client`). Log into Grafana at `localhost:3001` (default `admin/admin`) and add Prometheus as a data source.

Do this now, not in Part 4. You need working graphs to interpret the load test, and configuring monitoring under deadline pressure later is how evenings get lost.

2.4 — Frontend + combined load test

1. Build the login flow and a dashboard shell that renders real data from `/dashboard/home`.
2. Add both new services to the CI matrix.
3. Run k6 against Auth + Student + Dashboard together, ramping 100 → 500 → 750 virtual users.
4. Watch Grafana while the test runs — this is your first look at how the system behaves under combined load.

Exit criterion: logging in through the browser shows a populated dashboard, and Grafana shows live latency/error panels during a load test.

Part 3 — Transcript + Past Papers + Notification (22–25 September)

Three services in four days. Build Transcript first — it's the harder of the two data services.

3.1 — Transcript Service

`services/transcript-service/`. Uses the existing `transcript` schema and `transcript_svc` role.

Table	Holds
<code>grades</code>	<code>student_id</code> , <code>course_code</code> , <code>grade</code> , <code>semester</code> , <code>institution_id</code>
<code>gpa_snapshots</code>	Computed GPA per student per semester (cache, recomputed on grade change)

1. Add WeasyPrint (or your preferred PDF tool) to the Dockerfile for this service — it needs system dependencies, so check WeasyPrint's Docker install notes before assuming `pip install` alone is enough.
2. Endpoint `GET /transcript/:studentId/pdf` renders an HTML template to PDF and returns it.
3. On a new grade, publish a `grade.posted` event to RabbitMQ (`amqp://portal:changeme@rabbitmq:5672`) instead of calling Notification directly — this is the event-driven decoupling from the original plan.
4. Route: `PathPrefix(`/transcript`)`, same labels pattern as before.

3.2 — Past Papers Service

`services/pastpapers-service/`. Uses the `pastpapers` schema. Two storage pieces:

Storage	Holds
MinIO (add as a new docker-compose service)	The actual PDF files
Postgres (pastpapers schema)	<code>course</code> , <code>year</code> , <code>semester</code> , <code>uploader</code> , <code>tags</code> , MinIO object key

```
minio:
  image: minio/minio:latest
  container_name: portal-minio
  command: server /data --console-address ":9001"
  environment:
    MINIO_ROOT_USER: ${MINIO_USER:-portal_admin}
    MINIO_ROOT_PASSWORD: ${MINIO_PASSWORD:-changeme123}
  volumes:
    - minio-data:/data
  ports:
    - "9000:9000"
    - "9001:9001"
  networks:
    - portal-net
```

Remember to add `minio-data:` under the top-level `volumes:` key. Use Postgres full-text search (`tsvector`) on the metadata table for the search endpoint — that's sufficient for the deadline; don't add a dedicated search engine now.

1. Endpoints: `upload`, `search` by `course/year/semester`, `download`, `list`.
2. On upload, publish a `pastpaper.uploaded` event to RabbitMQ.
3. Route: `PathPrefix(`/pastpapers`)`.

3.3 — Notification Service (minimal, on purpose)

services/notification-service/. No database, no frontend-facing route — it only consumes RabbitMQ events.

1. Subscribe to `grade.posted` and `pastpaper.uploaded`.
2. Send one email per event using a single template each (any SMTP provider's free tier, or log to console if email setup would eat time you don't have).
3. Resist expanding this service. It is deliberately minimal — the cut-list in the plan document removes it first if you fall behind.

3.4 — Frontend and documentation

1. Transcript view/download page; Past Papers browse/upload/download page.
2. Swagger docs for both new services, written as you build — not retrofitted.
3. Add both to the CI matrix.

Exit criterion: a student can view/download a PDF transcript and upload/search/download a past paper, all through the gateway, with the notification firing on each event.

Part 4 — Full-System Load Test at 1,000 Users (26–28 September)

This produces the evidence your entire submission rests on. Budget real time here — treat a discovered bottleneck as expected work, not a crisis.

4.1 — Run the load generator from a separate machine

Never run k6 on the same box as the system under test — CPU contention between the generator and the system skews every number you record. Use a second computer, or a second free-tier cloud instance, pointed at your Oracle Cloud deployment's public IP.

4.2 — Ramp deliberately

Stage	Virtual users	Record
1	250	p50 / p95 / p99 latency, error rate
2	500	same, plus Grafana screenshot
3	750	same, plus Grafana screenshot
4	1,000	same — this is the number that matters most
5 (bonus)	3,000	only attempt once 1,000 is clean

4.3 — Diagnose whatever breaks

Watch Grafana *during* the run, not just after — the panel that saturates first tells you where to look. In likely order:

Symptom	Likely cause	Fix
Requests time out, Postgres errors spike	Connection pool exhausted	Check PgBouncer's DEFAULT_POOL_SIZE and MAX_CLIENT_CONN in docker-compose.yml — raise pool size or add read replicas if needed
Dashboard/BFF slows first	Missing or too-short cache TTL	Increase the Redis cache TTL on the dashboard aggregate from 2.3
One service's containers max out CPU	Under-replicated service	Scale it: see 4.4 below

4.4 — Scale horizontally

Remove the container_name: line from any service you plan to scale (Docker can't give multiple containers the same name), then:

```
docker compose up -d --scale auth-service=3 --scale student-service=3
```

Traefik's Docker provider detects the new replicas automatically and load-balances across them — no Traefik config changes needed.

4.5 — Save your evidence

1. Export the k6 summary report for each stage (JSON or HTML output).
2. Screenshot Grafana at the 1,000-user peak, showing latency and error-rate panels.
3. Note in writing what bottleneck you found and how you fixed it — this becomes part of your report and is genuinely good material for defence questions.

Decision point, 28 September. If 1,000 concurrent passes cleanly and you have time left, this is the only point in the schedule to consider a stretch service (Timetable is cheapest; Course & Enrolment is most impressive). If it does not pass yet, stop everything else and fix the bottleneck — the scalability claim is the project.

Part 5 — Documentation, Polish, Rehearsal (29–30 September)

5.1 — Feature freeze

No new functionality after 29 September, regardless of temptation. Everything from here is stabilising what exists.

5.2 — Write the report

Section	Content
Architecture	Final diagram matching what you actually built, not the original plan if they diverged
Database strategy	Schema-per-service rationale (Option A), with the actual schema
Storage split	Clouinary vs MinIO decision and why
Load-test results	The k6 reports and Grafana screenshots from Part 4, with your bottleneck-and-fix narrative
Deferred roadmap	The Post-deadline tier services from the services document — deliberate deferral reads as judgement

5.3 — UI polish pass

Go through every section — Dashboard, Auth flow, Transcript, Past Papers — for visual consistency. This is where your ByKenzel eye for visual work is a genuine advantage over a purely backend-focused submission.

5.4 — Record a backup demo video

Full walkthrough, screen-recorded, in case the live demo fails on the day. Do not skip this — it is one of the cheapest insurance policies available to you.

5.5 — Rehearse

Walk through the full demo out loud, at least twice. Prepare answers for the predictable questions:

Likely question	Core of your answer
Why microservices instead of a monolith?	Independent scaling and deployment per section — the original brief's incremental proof requirement
Why schema-per-service instead of database-per-service?	Same isolation in code, far less connection-pooling and ops overhead under time pressure
How do you prevent a lecturer silently altering a grade?	grade.posted events to RabbitMQ give you an audit trail — mention the Audit Service in your deferred roadmap if not built
What happens when one service goes down?	Event-driven decoupling via RabbitMQ — Notification being down doesn't block Transcript from posting a grade

Part 6 — Final Checks and Defence Day (1 October)

6.1 — Clean deployment proof

1. On the Oracle Cloud instance, from a completely fresh clone: `./setup-oracle-cloud.sh <your-repo-url>`
2. This proves the system builds from scratch, not just on your laptop with accumulated local state.

6.2 — Final route check

Manually verify every route through the gateway one last time: `/auth`, `/student`, `/dashboard`, `/transcript`, `/pastpapers`.

6.3 — Deliverables in hand

Item	✓
Working deployed system reachable through the gateway	■
k6 reports at 250 / 500 / 750 / 1,000 concurrent users	■
Grafana screenshots from the 1,000-user run	■
Final architecture diagram and schema documentation	■
Swagger/OpenAPI docs per service	■
Written report, including the deferred-services roadmap	■
Backup demo video	■
Clean-clone deployment verified on Oracle Cloud	■

That's the full path from an empty pipeline to a defended project. If you get stuck on any single step — a service won't start, a load test result looks wrong, a Traefik route 404s — come back with exactly what you ran and what came back, the same way we solved the Docker socket issue, and we'll work through it the same way.

Companion documents: [student-portal-pipeline.zip](#) (the code this guide walks through) and [student-portal-services-and-plan.pdf](#) (the full service catalogue and risk register this guide implements).